Solution: Minimum Deletions & Insertions to Transform a String into another

Problem Statement

Given strings s1 and s2, we need to transform s1 into s2 by deleting and inserting characters. Write a function to calculate the count of the minimum number of deletion and insertion operations.

Example 1:

```
Input: s1 = "abc"
       s2 = "fbc"
Output: 1 deletion and 1 insertion.
Explanation: We need to delete {'a'} and insert {'f'} to s1 to transform
it into s2.
```

Example 2:

```
Input: s1 = "abdca"
       s2 = "cbda"
Output: 2 deletions and 1 insertion.
Explanation: We need to delete {'a', 'c'} and insert {'c'} to s1 to
transform it into s2.
```

Example 3:

```
Input: s1 = "passport"
       s2 = "ppsspt"
Output: 3 deletions and 1 insertion
Explanation: We need to delete {'a', 'o', 'r'} and insert {'p'} to s1 to
transform it into s2.
```

Solution

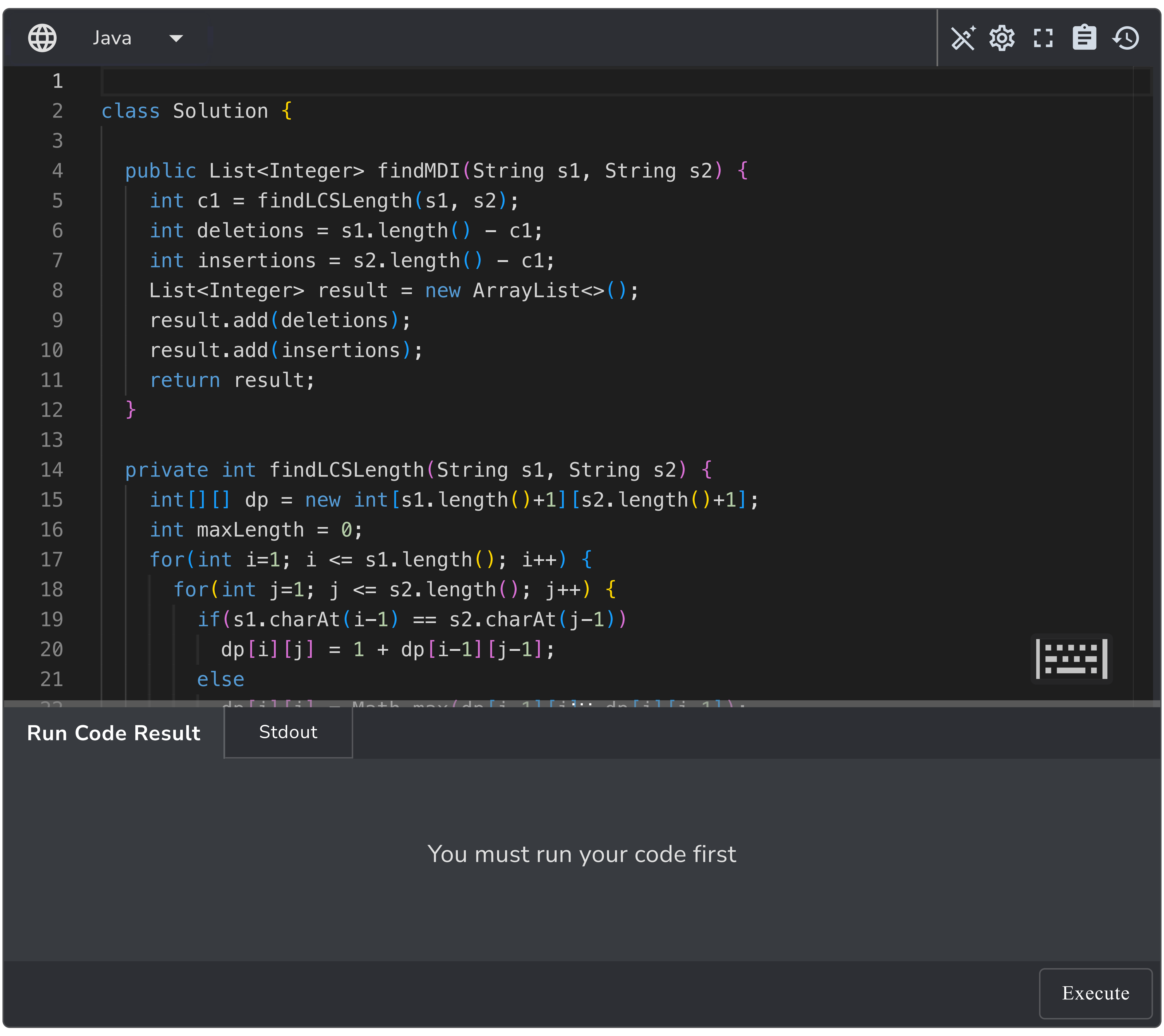
This problem can easily be converted to the `"Longest Common Subsequence" (LCS)`. If we can find the LCS of the two input strings, we can easily find how many characters we need to insert and delete from s1. Here is how we can do this:

1. Let's assume `len1` is the length of s1 and `len2` is the length of s2.
2. Now let's assume `c1` is the length of LCS of the two strings s1 and s2.
3. To transform s1 into s2, we need to delete everything from s1 which is not part of LCS, so minimum deletions we need to perform from s1 => `len1 - c1`
4. Similarly, we need to insert everything in s1 which is present in s2 but not part of LCS, so minimum insertions we need to perform in s1 => `len2 - c1`

Let's jump directly to the bottom-up dynamic programming solution:

Bottom-up Dynamic Programming

Here is the code for our bottom-up dynamic programming approach:



```

class Solution {

  public List<Integer> findMDI(String s1, String s2) {
    int c1 = findLCSLength(s1, s2);
    int deletions = s1.length() - c1;
    int insertions = s2.length() - c1;
    List<Integer> result = new ArrayList<>();
    result.add(deletions);
    result.add(insertions);
    return result;
  }

  private int findLCSLength(String s1, String s2) {
    int[][] dp = new int[s1.length()+1][s2.length()+1];
    int maxLength = 0;
    for(int i=1; i <= s1.length(); i++) {
      for(int j=1; j <= s2.length(); j++) {
        if(s1.charAt(i-1) == s2.charAt(j-1))
          dp[i][j] = 1 + dp[i-1][j-1];
        else
```

The time and space complexity of the above algorithm is $O(m*n)$, where 'm' and 'n' are the lengths of the two input strings.

← Previous
Minimum Deletions & Insertions To Transform A String Into Another

Next →
Longest Increasing Subsequence

☐ Mark as Completed